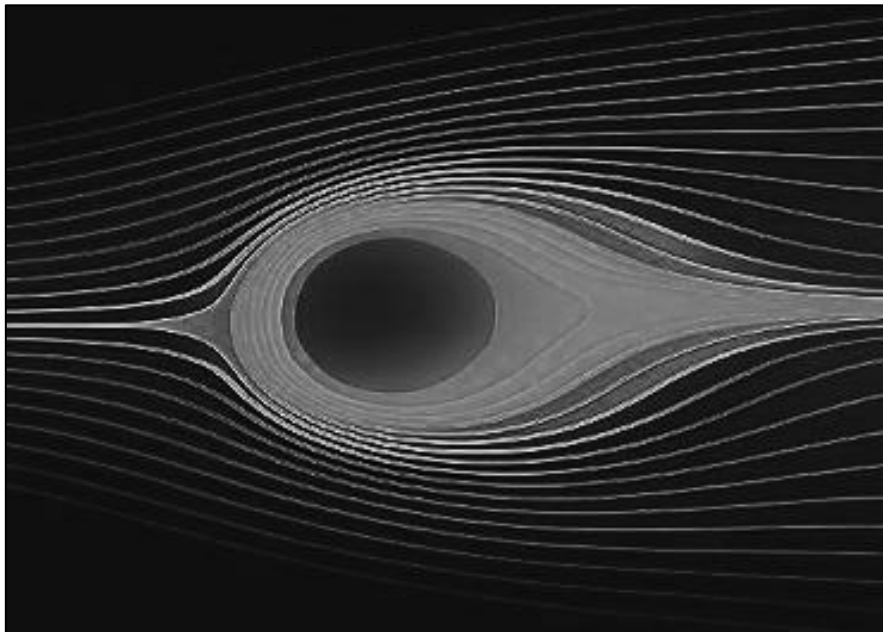


Επίλυση Ασκήσεων Μαθήματος

Υπολογιστική Ρευστομηχανική

3^η Σειρά Ασκήσεων



Ονοματεπώνυμο: Αναστάσιος Μπατατέγας

Αριθμός Μητρώου: 09121431

Εξάμηνο: 9^ο

Σχολή: ΕΜΦΕ

Email: tasosmntdew@gmail.com , ge21431@mail.ntua.gr

Διδάσκων: Ε. Ρετσίνης

Άσκηση 3.1

Σε σωλήνα διαμέτρου $D = 1$ m μεταφέρεται ορυκτέλαιο (πυκνότητας $\rho = 870$ kg/m³ και συνεκτικότητας $\mu = 0.104$ kg · m⁻¹ · s⁻¹) με παροχή $Q = 0.16$ m³/s. Ζητούνται:

1. Υλοποιήστε στην γλώσσα προγραμματισμού Matlab την αριθμητική επίλυση του προβλήματος μόνιμης, πλήρως ανεπτυγμένης (ομοιόμορφης), στρωτής ροής υπό σταθερή κλίση πίεσης στον εν λόγω σωλήνα για τον προσδιορισμό της ακτινικής κατανομής ταχύτητας κατά την κύρια διεύθυνση της ροής. Τα αποτελέσματα εκτός από την αριθμητική τους μορφή να παρουσιαστούν και σε μορφή κατάλληλου διαγράμματος. (Μπορείτε να επιλύσετε το αλγεβρικό σύστημα εξισώσεων με άμεση ή επαναληπτική μέθοδο.)
2. Επαληθεύστε την τιμή της μέσης ταχύτητας που προκύπτει θεωρητικά με την τιμή που προκύπτει ολοκληρώνοντας αριθμητικά τα αποτελέσματα για την κατανομή της ταχύτητας.
3. Αξιολογείστε την επίδραση της μεταβολής του υπολογιστικού πλέγματος στην αριθμητική λύση του προβλήματος (εκτελέστε τον υλοποιημένο κώδικα Matlab επιλέγοντας κάθε φορά διαφορετικό πλήθος υπολογιστικών κόμβων). Παρουσιάστε τις αριθμητικές λύσεις που προκύπτουν υπό μορφή διαγραμμάτων. Σχολιάστε τα αποτελέσματα.

Λύση

Προς επίλυση του προβλήματος, ακολουθούμε την ανάλυση των Σημειώσεων. Εντός του μήκους ανάπτυξης της ροής, τα οριακά στρώματα αναπτύσσονται συνεχώς, έως ότου το πάχος τους να γίνει ίσο με την ακτίνα του σωλήνα, φτάνοντας επί του άξονά του. Μετά το πέρας του μήκους ανάπτυξης, η κατανομή της ταχύτητας σταθεροποιείται σε κάθε θέση κατά μήκος του άξονα x του σωλήνα, ενώ η κλίση της πίεσης στην κατάσταση αυτή είναι επίσης σταθερή (λόγω μηδενικής επιτάχυνσης του ρευστού). Προς περιγραφή της κίνησης του ρευστού χρησιμοποιούνται κυλινδρικές συντεταγμένες (r, φ, x) , όπου r, φ η πολική ακτίνα και γωνία, αντίστοιχα, και x συντεταγμένη κατά μήκος του άξονα του σωλήνα. Η ακτίνα του σωλήνα είναι ίση με:

$$R = D/2 = 0.5 \text{ m} \quad (3.1)$$

Λόγω συμμετρίας της μόνιμης ροής, είναι $u_r = u_\varphi = 0$, λόγω της μονιμότητας της ροής $\dot{u}_x = 0$, λόγω πλήρους ανάπτυξης και ομοιομορφίας της είναι $du_x/dx = 0$ και $dp/dx = \text{const.}$, ενώ λόγω πολικής συμμετρίας της $du_x/d\varphi = 0$. Επομένως, το πεδίο ταχύτητας του ρευστού χαρακτηρίζεται από την μόνη μη μηδενική αξονική συνιστώσα του $u_x = u_x(r)$, και οι εξισώσεις Navier – Stokes αποδίδουν (έπειτα από εφαρμογή των συνοριακών συνθηκών) το Π.Σ.Σ. (βλ. και Σημειώσεις):

$$\left\{ \begin{array}{l} \frac{1}{r} \frac{du_x(r)}{dr} + \frac{d^2 u_x(r)}{dr^2} = \frac{1}{\mu} \frac{dp(x)}{dx} \\ r = R : u_x(r = R) = 0 \\ r = 0 : \left. \frac{du_x(r)}{dr} \right|_{r=0} = 0 \end{array} \right\} \quad (3.2)$$

όπου $dp/dx = \text{const.}$. Το Π.Σ.Σ. (3.2) επιδέχεται αναλυτική («θεωρητική») λύση την λεγόμενη *λύση ροής Poiseuille*:

$$u_x(r) = -\frac{R^2 - r^2}{4\mu} \frac{dp(x)}{dx} \quad (3.3)$$

με μέγιστη ταχύτητα επί του άξονα του σωλήνα:

$$u_{\max} \equiv u_x(r = 0) = -\frac{R^2}{4\mu} \frac{dp(x)}{dx} \quad (3.4)$$

Καθιστώντας το Π.Σ.Σ. αδιάστατο, «μετρώντας» την πολική ακτίνα r σε μονάδες ακτίνας R του σωλήνα, και την ταχύτητα $u_x(r)$ σε μονάδες της μέσης χωρικά ταχύτητας V_{average} , το (3.2) καθίσταται στην μορφή:

$$\left\{ \begin{array}{l} \frac{1}{r} \frac{du}{dr} + \frac{d^2u}{dr^2} = -8, \quad 0 \leq r \leq 1 \\ r = 1 : u(r = 1) = 0 \quad (\text{Σ. Σ. Dirichlet}) \\ r = 0 : \left. \frac{du(r)}{dr} \right|_{r=0} = 0 \quad (\text{Σ. Σ. Newmann}) \end{array} \right\} \quad (3.5)$$

υπό τον συμπυκνόμενο συμβολισμό $r \equiv r/R$, $u = u(r) \equiv u_x(r)/V_{\text{average}}$, όπου η μέση χωρικά ταχύτητα ισούται με:

$$V_{\text{average}} \equiv \frac{1}{E} \int u_x(r) dE = -\frac{R^2}{8\mu} \frac{dp(x)}{dx} \equiv \frac{u_{\max}}{2} = \frac{Q}{E} = \frac{4Q}{\pi D^2} \quad (3.6)$$

Επιπλέον, η λύση ροής Poiseuille (3.3), σε αδιάστατη μορφή καθίσταται ίση με:

$$u(r) = 2(1 - r^2), \quad r \equiv r/R, \quad u(r) \equiv u_x(r)/V_{\text{average}} \quad (3.7)$$

Στα επόμενα, το Π.Σ.Σ. (3.5) επιλύεται με την μέθοδο πεπερασμένων διαφορών. Αρχικά, το πεδίο ορισμού $r \in [0,1]$ διακριτοποιείται μέσω του εξής υπολογιστικού πλέγματος, N υπολογιστικών κόμβων $\{r_1, r_2, \dots, r_N\}$:

$$\left\{ r_1 = 0, \quad r_i = r_{i-1} + \Delta r = \frac{i}{N-1}, \quad \Delta r \equiv \frac{1}{N-1}, \quad i = 2, \dots, N-1 \right\} \quad (3.8)$$

Στην συνέχεια, η διαφορική εξίσωση του Π.Σ.Σ. (3.5) διακριτοποιείται σε εξίσωση πεπερασμένων διαφορών. Για τους εσωτερικούς κόμβους $i = 2, \dots, N-1$, τόσο η πρώτη, όσο και η δεύτερη παράγωγος αντικαθίστανται από τις αντίστοιχες εκφράσεις κεντρικών διαφορών, ακρίβειας 2^{ης} τάξης:

$$\frac{du}{dr} = \frac{u_{i+1} - u_{i-1}}{2\Delta r}, \quad \frac{d^2u}{dr^2} = \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta r^2}, \quad i = 2, \dots, N-1 \quad (3.9)$$

όπου αξιοποιείται ο συμβολισμός $u_j \equiv u(r_j)$, $j \in \{1, \dots, N\}$. Αντικατάσταση των (3.9) στην διαφορική εξίσωση του (3.5) συνεπάγεται ότι για τους εσωτερικούς κόμβους προκύπτει το $(N-2) \times (N-2)$ τριδιαγώνιο γραμμικό σύστημα:

$$\left\{ \left(1 - \frac{\Delta r}{2r_i} \right) u_{i-1} - 2u_i + \left(1 + \frac{\Delta r}{2r_i} \right) u_{i+1} = -8(\Delta r)^2, \quad i = 2, \dots, N-1 \right\} \quad (3.10a)$$

Η Σ.Σ. τύπου Dirichlet στο $r = 1$ διακριτοποιείται αυτομάτως, ενώ για την Σ.Σ. τύπου Newmann στο $r = 0$ η πρώτη παράγωγος αντικαθίσταται από την ακραία διαφορά, ακρίβειας 1^{ης} τάξης. Τελικά λαμβάνουμε ότι οι διακριτοποιημένες Σ.Σ. είναι οι εξής:

$$\begin{cases} r = 0: & u_1 = u_2 \\ r = 1: & u_N = 0 \end{cases} \quad (3.10b)$$

Το σύστημα των (3.10a) και (3.10b) είναι τριδιαγώνιο, όπως φαίνεται γράφοντάς το στην παρακάτω μορφή:

$$\left\{ \begin{array}{c} u_1 - u_2 = 0 \\ \vdots \\ \left(1 - \frac{\Delta r}{2r_i} \right) u_{i-1} - 2u_i + \left(1 + \frac{\Delta r}{2r_i} \right) u_{i+1} = -8(\Delta r)^2, \quad i = 2, \dots, N-1 \\ \vdots \\ u_N = 0 \end{array} \right\} \quad (3.11)$$

και η επίλυσή του, μέσω του κώδικα Matlab που υλοποιείται παρακάτω, αποδίδει προσέγγιση ακρίβειας 1^{ης} τάξης της αναλυτικής λύσης (3.3), ισοδύναμα της (3.7).

1. Προς επίλυση του τριδιαγώνιου συστήματος (3.11) αξιοποιείται η άμεση μέθοδος του αλγορίθμου Thomas. Ο τριδιαγώνιος πίνακας του συστήματος ορίζεται αξιοποιώντας τρία διανύσματα τύπου $N \times 1$, τα οποία αντιπροσωπεύουν την κάτω (**a**), κύρια (**b**) και άνω (**c**) διαγώνιο, και τα οποία ορίζονται, βάσει της (3.10), ως εξής:

$$\mathbf{a} \equiv \begin{bmatrix} 0 \\ 1 - \frac{\Delta r}{2r_2} \\ \vdots \\ 1 - \frac{\Delta r}{2r_i} \\ \vdots \\ 1 - \frac{\Delta r}{2r_{N-1}} \\ 0 \end{bmatrix}_{N \times 1}, \quad \mathbf{b} \equiv \begin{bmatrix} 1 \\ -2 \\ \vdots \\ -2 \\ 1 \end{bmatrix}_{N \times 1}, \quad \mathbf{c} \equiv \begin{bmatrix} -1 \\ 1 + \frac{\Delta r}{2r_2} \\ \vdots \\ 1 + \frac{\Delta r}{2r_i} \\ \vdots \\ 1 + \frac{\Delta r}{2r_{N-1}} \\ 0 \end{bmatrix}_{N \times 1} \quad (3.12)$$

Το πρώτο στοιχείο του **a** και το τελευταίο στοιχείο του **c** τίθενται εξ ορισμού («δια χειρός») ίσα με το 0 προς διατήρηση κοινών διαστάσεων για τα υπό χρήση διανύσματα των τριών διαγώνιων του τριδιαγώνιου πίνακα του συστήματος (3.11).

Στην συνέχεια, υλοποιείται σε γλώσσα Matlab ο αλγόριθμος Thomas, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab συνάρτησης (m-function file) με όνομα “ThomasAlgorithmTriInput.m”, και παρατίθεται παρακάτω. Η εν λόγω συνάρτηση υλοποίησης του αλγορίθμου Thomas, αποτελεί ελαφρώς τροποποιημένη εκδοχή της αντίστοιχης επεξηγημένης συνάρτησης της Άσκησης 1.1 της 1^{ης} Σειράς Ασκήσεων (η οποία παρατίθεται παρακάτω για λόγους πληρότητας), ώστε να δέχεται ως όρισμά της μόνον τις διαγώνιους (**a**, **b**, **c**) του πίνακα του τριδιαγώνιου συστήματος,

και όχι ολόκληρο τον πίνακα A αυτού. Κατά τα άλλα, η «γενική λογική» υλοποίησης του αλγορίθμου καθώς και ο έλεγχος των δεδομένων εισόδου, γίνεται με κοινό τρόπο, και για τον λόγο αυτό, οι αντίστοιχες επεξηγήσεις παραλείπονται. Ο πλήρης πηγαίος κώδικας, τόσο της επικείμενης συνάρτησης “ThomasAlgorithmTriInput.m”, όσο και της συνάρτησης “ThomasAlgorithm.m” της Άσκησης 1.1, παρουσιάζονται παρακάτω.

MATLAB FUNCTION M-FILE (ThomasAlgorithm.m):

```
function [x] = ThomasAlgorithm(A,d)
% Function ThomasAlgorithm.m
%
% =====
% cf. Exercise 1.1, 1st Problem Set
% =====
%
%-----
% Solution of tridiagonal linear system A*x = d.
% Input : - square tridiagonal matrix A of coefficients (n x n).
%         - const. vector d (n x 1).
% Output: - vector x = solution of (Ax=d) (n x 1).
%-----

% --- System dimension & solution definition ---
n = length(d);
x = zeros(n,1);

% --- Input validity ---
% Ensure d is in column form :
d = d(:);
% Ensure A and d are properly sized :
[m1,m2]=size(A);
if(m1 ~= m2)
    error('Matrix A is not square (n x n).');
elseif(m1 ~= n)
    error('Matrix A and vector d are incompatible.');
```

```
end
% Ensure A is a tridiagonal matrix:
if(~isbanded(A,1,1))
    error('Matrix A is not tridiagonal.');
```

```
end

% --- Extract from A only the non-zero diagonals ---
% Form vector a (n x 1) of lower-diagonal elements:
a = zeros(n,1);
a(2:n) = diag(A,-1);
% Form vector b (n x 1) of diagonal elements:
b = zeros(n,1);
b = diag(A,0);
% Form vector c (n x 1) of upper-diagonal elements:
c = zeros(n,1);
c(1:n-1) = diag(A,1);

% --- Thomas' method implementation ---
% --- 1. Forward Elimination Stage ---
c(1) = c(1)/b(1);
d(1) = d(1)/b(1);
```

```
% Loop over vector c:
for i = 2 : n-1
    denominator = b(i)-a(i)*c(i-1);
    if(denominator==0)
        error('Matrix A is singular or has bad condition.');
```

end

```
    c(i) = c(i)/denominator;
end
% Loop over vector d:
for i = 2 : n
    denominator = b(i)-a(i)*c(i-1);
    if(denominator==0)
        error('Matrix A is singular or has bad condition.');
```

end

```
    d(i) = (d(i)-a(i)*d(i-1))/denominator;
end
x(n) = d(n);
% --- 2. Backward Substitution Stage ---
for i = n-1 : -1 : 1
    x(i) = d(i) - c(i)*x(i+1);
end
end
```

MATLAB FUNCTION M-FILE (ThomasAlgorithmTriInput.m):

```
function [x] = ThomasAlgorithmTriInput(a,b,c,d)
% Function ThomasAlgorithmTriInput.m
%-----
% Solution of tridiagonal linear system A*x = d, A=diag(a,b,c)
% Input : - diagonal vectors a,b,c of matrix A          (n x 1).
%          - const. vector d                            (n x 1).
% Output: - vector x = solution of (Ax=d)               (n x 1).
%-----

% --- System dimension & solution definition ---
n = length(d);
x = zeros(n,1);

% --- Input validity ---
% Ensure d is in column form :
d = d(:);
% Ensure a,b,c and d are properly sized :
if(length(a) ~= n || a(1) ~= 0)
    error('Lower diagonal a is not (n x 1) or a(1) isnot 0.');
```

elseif(length(b) ~= n)

```
    error('Diagonal b is not (n x 1).');
```

elseif(length(c) ~= n || c(n) ~= 0)

```
    error('Upper diagonal c is not (n x 1) or c(n) isnot 0.');
```

end

% --- Thomas' method implementation ---

% --- 1. Forward Elimination Stage ---

```
c(1) = c(1)/b(1);
d(1) = d(1)/b(1);
% Loop over vector c:
for i = 2 : n-1
    denominator = b(i)-a(i)*c(i-1);
    if(denominator==0)
```

```
        error('Matrix A is singular or has bad condition.');
```

```
    end
    c(i) = c(i)/denominator;
end
% Loop over vector d:
for i = 2 : n
    denominator = b(i)-a(i)*c(i-1);
    if(denominator==0)
        error('Matrix A is singular or has bad condition.');
```

```
    end
    d(i) = (d(i)-a(i)*d(i-1))/denominator;
end
x(n) = d(n);
% --- 2. Backward Substitution Stage ---
for i = n-1 : -1 : 1
    x(i) = d(i) - c(i)*x(i+1);
end
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του συστήματος (3.10) πραγματοποιείται άμεσα. Στο παρακάτω αρχείο (m-script file), παρουσιάζεται η επίλυση του εν λόγω συστήματος για αριθμό κόμβων υπολογιστικού πλέγματος ίσο με $N = 101$, τιμή η οποία, αν και δεν παρέχει ιδιαίτερα υψηλή ακρίβεια, εντούτοις χρησιμοποιείται ώστε να γίνει φανερή η σχέση της αναλυτικής λύσης με την αντίστοιχη υπολογιζόμενη αριθμητικά. Επιπλέον, η αριθμητική επίλυση του εν λόγω συστήματος, γίνεται, προς επαλήθευση και έλεγχο του κώδικα, και με χρήση της μεθόδου “A\d” της Matlab, καθώς και με χρήση της συνάρτησης του αλγορίθμου Thomas της Άσκησης 1.1 της 1^{ης} Σειράς Ασκήσεων (η οποία παρατίθεται παραπάνω). Η ταύτιση, ή μη, της αριθμητικά υπολογιζόμενης λύσης με την μέθοδο “Thomas(a,b,c,d)” σε σχέση με τις λύσεις που προκύπτουν από τις μεθόδους “A\d” και “Thomas(A,d)”, ελέγχεται έως και το 8^ο δεκαδικό ψηφίο (10^{-8}). Τα αποτελέσματα επίλυσης, καθώς και η αντίστοιχη αριθμητική και αναλυτική λύση, παρουσιάζονται και γραφικά.

MATLAB SCRIPT M-FILE (ThomasAlgorithmTriInputEx3_1_Script.m):

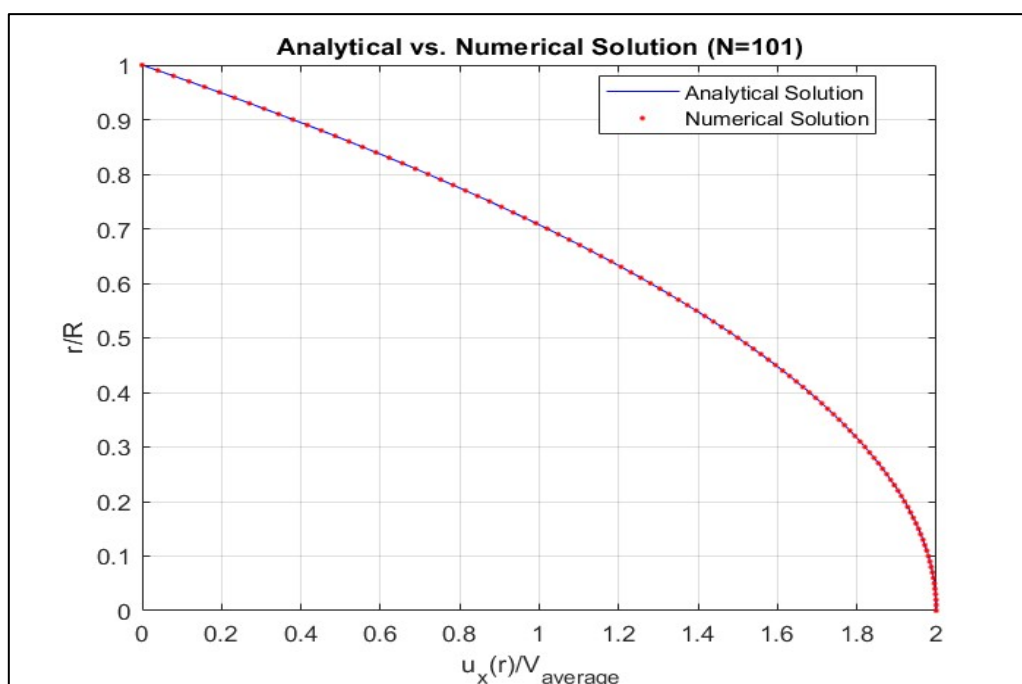
```
clear
clc
% --- CFD Exercise 3.1 ---
% --- Part 1 ---
% Solution script (using ThomasAlgorithmTriInput.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of number of nodes N ---
N = 101
% --- Definition of computational grid ---
r = linspace(0,1,N);
dr = 1/(N-1);
% --- Definition of matrix A=diag(a,b,c) and d ---
% Initializations:
a = zeros(N,1) ;
b = -2*ones(N,1); b(1) = 1; b(N) = 1;
c = zeros(N,1) ; c(1) = -1; c(N) = 0;
```

```
d = zeros(N,1) ;
% Definitions:
for i = 2:N-1
    a(i) = 1 - dr/(2*r(i));
    c(i) = 1 + dr/(2*r(i));
    d(i) = -8*(dr)^2;
end
%
% -----
% --- SOLUTION OF B.V.P. ---
% -----
% --- Apply Thomas algorithm and get solution ---
u = ThomasAlgorithmTriInput(a,b,c,d);
%
% --- Check validity of solution ---
% Definition of auxiliary matrices for proper operation
% of "spdiags" function:
aa = zeros(N,1);
cc = zeros(N,1);
for i = 1:N-1
    aa(i) = a(i+1); cc(i+1) = c(i);
end
% System's matrix "reconstruction":
A = spdiags([aa b cc],-1:1,N,N);
A = full(A);
% Solve system by "A\d" method:
u1 = A\d;
if(abs(u-u1)<10^-8)
    disp('Solution via Thomas(a,b,c,d) algorithm and via A\b match!')
end
% Solve system by "Thomas(A,d)" method:
u1T = ThomasAlgorithm(A,d);
if(abs(u-u1T)<10^-8)
    disp('Solution via Thomas(A,d) and via Thomas(a,b,c,d) match')
end
%
% --- Definition of analytical solution function handle ---
u_analytical = @(r) 2*(1-(r).^2);
% --- Definition of analytical solution at partition nodes ---
u_anal = u_analytical(r);
%
% --- Definition of table for data output ---
i = (1:N)';
T = table(i, r(:), u(:), u_anal(:), abs(u(:)-u_anal(:)));
T = renamevars(T,"Var2","r");
T = renamevars(T,"Var3", "u_numerical");
T = renamevars(T,"Var4", "u_analytical");
T = renamevars(T,"Var5", "abs error");
disp(T(1:10,:));
disp('.....');
disp(T(end-9:end,:));
%
% --- Plot numerical and analytical solution ---
figure;
plot(u_anal, r, 'b-', 'LineWidth', 0.5); hold on;
plot(u, r, 'r.', 'LineWidth', 0.5);
grid on;
ylabel('r/R');
xlabel('u_{x}(r)/V_{average}');
```

```
title(['Analytical vs. Numerical Solution (N=', num2str(N), ')']);
legend('Analytical Solution', 'Numerical Solution', 'Location', 'Best');
%
% End of script.
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του συστήματος (3.11) μέσω του αλγορίθμου Thomas, σε μορφή εκτύπωσης που καθορίζεται από την συνάρτηση `table`. Προς σύμπτυξη της παρουσίασης των δεδομένων, παρουσιάζονται μόνον οι δέκα πρώτες και δέκα τελευταίες γραμμές του πίνακα, όπου περιέχονται οι αριθμητικά υπολογιζόμενες τιμές, καθώς και οι αναλυτικές τιμές, σε κάθε κόμβο του υπολογιστικού πλέγματος. Η αντίστοιχη γραφική παράσταση του αδιάστατου προφίλ ταχύτητας κατά την διατομή του σωλήνα φαίνεται στο Διάγραμμα I παρακάτω.

Command Window				
N =				
101				
Solution via Thomas(a,b,c,d) algorithm and via A\b match!				
Solution via Thomas(A,d) and via Thomas(a,b,c,d) match				
i	r	u_numerical	u_analytical	abs error
1	0	1.9993	2	0.00065687
2	0.01	1.9993	1.9998	0.00045687
3	0.02	1.9988	1.9992	0.0003902
4	0.03	1.9978	1.9982	0.0003502
5	0.04	1.9965	1.9968	0.00032163
6	0.05	1.9947	1.995	0.00029941
7	0.06	1.9925	1.9928	0.00028123
8	0.07	1.9899	1.9902	0.00026584
9	0.08	1.9869	1.9872	0.00025251
10	0.09	1.9836	1.9838	0.00024074
.....				
i	r	u_numerical	u_analytical	abs error
92	0.91	0.34379	0.3438	9.431e-06
93	0.92	0.30719	0.3072	8.3381e-06
94	0.93	0.27019	0.2702	7.257e-06
95	0.94	0.23279	0.2328	6.1875e-06
96	0.95	0.19499	0.195	5.1293e-06
97	0.96	0.1568	0.1568	4.0822e-06
98	0.97	0.1182	0.1182	3.0459e-06
99	0.98	0.079198	0.0792	2.0203e-06
100	0.99	0.039799	0.0398	1.005e-06
101	1	0	0	0



Διάγραμμα I.

2. Δεδομένης της σχέσης (3.6) η αναλυτική τιμή για την μέση ταχύτητα επί της διατομής του σωλήνα ροής ισούται με:

$$V_{\text{average}}^{\text{anal.}} \equiv \frac{Q}{E} = \frac{4Q}{\pi D^2} \cong 0.2037 \text{ m/s} \quad (3.13)$$

δεδομένων των τιμών παροχής ($Q = 0.16 \text{ m}^3/\text{s}$) και διαμέτρου ($D = 1 \text{ m}$) σωλήνα. Επιπλέον, μπορεί να ελεγχθεί το κατά πόσο η υπό μελέτη ροή είναι πράγματι στρωτή, υπολογίζοντας τον λόγο Reynolds:

$$\text{Re} = \frac{V_{\text{average}}^{\text{anal.}} D \rho}{\mu} \cong 1705 < 2000 \quad (3.14)$$

Επομένως, η υπό μελέτη ροή είναι πράγματι στρωτή, και η όλη ανάλυση ευσταθεί. Από την αριθμητική επίλυση του παραπάνω εδαφίου για το εν λόγω πρόβλημα ρευστομηχανικής, έχει προκύψει μία προσέγγιση για την αδιάστατη κατανομή της ταχύτητας ροής του ορυκτελαίου, συναρτήσει της επίσης αδιάστατης πολικής ακτίνας επί της διατομής του σωλήνα ροής, ήτοι η συνάρτηση $u = u(r)$, η θεωρητική έκφραση της οποίας δίδεται από την σχέση (3.7). Επομένως, αριθμητικός υπολογισμός για την τιμή της μέσης χωρικά ταχύτητας, με κανονικές μονάδες, δεν μπορεί να προκύψει. Ωστόσο, για λόγους επαλήθευσης της ορθότητας της προκύπτουσας αριθμητικής λύσης, μπορούμε να υπολογίσουμε το ολοκλήρωμα της αδιάστατης μέσης ταχύτητας που προκύπτει από αδιαστατοποίησης του ολοκληρώματος της (3.6), ήτοι του:

$$V_{\text{average}} \equiv \frac{1}{E} \int u_x(r) dE = \frac{1}{\pi R^2} \int_0^R u_x(r) 2\pi r dr$$

Εισάγοντας τις αδιάστατες ποσότητες $u(r) \equiv u_x(r)/V_{\text{average}}$ και $r \equiv r/R$ στην τελευταία σχέση, προκύπτει η τιμή της αδιάστατης μέσης χωρικά ταχύτητας:

$$\bar{U}_{\text{average}} \equiv 2 \cdot \int_0^1 u(r) r dr = 1 \quad (3.15)$$

Επομένως, προς επαλήθευση της ορθότητας της αριθμητικής αδιάστατης λύσης $u(r)$ που προκύπτει από την ανάλυση και τον κώδικα του προηγούμενου εδαφίου, αρκεί να πραγματοποιηθεί αριθμητική ολοκλήρωση της συνάρτησης $2u(r)r$ στο διάστημα $[0,1]$ και το αποτέλεσμα να συγκριθεί με την μονάδα. Ισοδύναμα, πολλαπλασιάζοντας το αποτέλεσμα της παραπάνω ολοκλήρωσης με την αναλυτική τιμή $V_{\text{average}}^{\text{anal.}}$, προκύπτει μία «αριθμητική» τιμή, $V_{\text{average}}^{\text{num.}}$, για την μέση ταχύτητα, η οποία συγκρίνεται με την αναλυτική τιμή, $V_{\text{average}}^{\text{anal.}}$.

Τα παραπάνω υλοποιούνται στο παρακάτω αρχείο (m-script file), το οποίο αποτελεί μία «επέκταση» του αντίστοιχου αρχείου m-script που παρατίθεται στο προηγούμενο εδάφιο (για τον λόγο αυτό έχει και το ίδιο όνομα). Η μέση χωρικά ταχύτητα, υπολογίζεται τόσο αναλυτικά, εισάγοντας τις αντίστοιχες παραμέτρους παροχής και διαμέτρου σωλήνα, και αξιοποιώντας την (3.13), όσο και «αριθμητικά», υπό την έννοια που επεξηγήθηκε παραπάνω. Η εμπλεκόμενη αριθμητική ολοκλήρωση

πραγματοποιείται μέσω της συνάρτησης `trapz(r,w)` της Matlab, η οποία εκτελεί την μέθοδο του τραπεζίου προς ολοκλήρωση των εισαγόμενων ορισμάτων της.

MATLAB SCRIPT M-FILE (ThomasAlgorithmTriInputEx3_1_Script.m):

```
clear
clc

% --- CFD Exercise 3.1 ---
% --- Part 1 ---
% Solution script (using ThomasAlgorithmTriInput.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of number of nodes N ---
N = 101
% --- Definition of computational grid ---
r = linspace(0,1,N);
dr = 1/(N-1);
% --- Definition of matrix A=diag(a,b,c) and d ---
% Initializations:
a = zeros(N,1) ;
b = -2*ones(N,1); b(1) = 1; b(N) = 1;
c = zeros(N,1) ; c(1) = -1; c(N) = 0;
d = zeros(N,1) ;
% Definitions:
for i = 2:N-1
    a(i) = 1 - dr/(2*r(i));
    c(i) = 1 + dr/(2*r(i));
    d(i) = -8*(dr)^2;
end
%
% -----
% --- SOLUTION OF B.V.P. ---
% -----
% --- Apply Thomas algorithm and get solution ---
u = ThomasAlgorithmTriInput(a,b,c,d);
%
% --- Check validity of solution ---
% Definition of auxiliary matrices for proper operation
% of "spdiags" function:
aa = zeros(N,1);
cc = zeros(N,1);
for i = 1:N-1
    aa(i) = a(i+1); cc(i+1) = c(i);
end
% System's matrix "reconstruction":
A = spdiags([aa b cc],-1:1,N,N);
A = full(A);
% Solve system by "A\d" method:
u1 = A\d;
if(abs(u-u1)<10^-8)
    disp('Solution via Thomas(a,b,c,d) algorithm and via A\b match!')
end
% Solve system by "Thomas(A,d)" method:
u1T = ThomasAlgorithm(A,d);
if(abs(u-u1T)<10^-8)
```

```

    disp('Solution via Thomas(A,d) and via Thomas(a,b,c,d) match')
end
%
% --- Definition of analytical solution function handle ---
u_analytical = @(r) 2*(1-(r).^2);
% --- Definition of analytical solution at partition nodes ---
u_anal = u_analytical(r);
%
% --- Definition of table for data output ---
i = (1:N)';
T = table(i, r(:), u(:), u_anal(:), abs(u(:)-u_anal(:)));
T = renamevars(T,"Var2","r");
T = renamevars(T,"Var3", "u_numerical");
T = renamevars(T,"Var4", "u_analytical");
T = renamevars(T,"Var5", "abs error");
disp(T(1:10,:));
disp(' ..... ');
disp(T(end-9:end,:));
%
% --- Plot numerical and analytical solution ---
figure;
plot(u_anal, r, 'b-', 'LineWidth', 0.5); hold on;
plot(u, r, 'r.', 'LineWidth', 0.5);
grid on;
ylabel('r/R');
xlabel('u_{x}(r)/V_{average}');
title(['Analytical vs. Numerical Solution (N=', num2str(N), ')']);
legend('Analytical Solution', 'Numerical Solution', 'Location', 'Best');
%
% -----
% --- Part 2 ---
% --- Calculation of spatially average velocity ---
% --- 1. Analytical calculation ---
Q = 0.16;
D = 1;
Vav_anal = 4*Q/(pi*D^2)
%
% --- 2. Numerical calculation ---
% Definition of integrant:
w = u(:) .* r(:);
% "Numerical" average speed calculation:
Vav_num = 2*Vav_anal*trapz(r,w)
%
% End of script.

```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του συστήματος (3.11) μέσω του αλγορίθμου Thomas, σε μορφή εκτύπωσης που καθορίζεται από την συνάρτηση `table`, καθώς και το Διάγραμμα I, όπως ακριβώς και στο προηγούμενο εδάφιο. Επιπλέον, λαμβάνουμε την αναλυτική και την «αριθμητική» τιμή της μέσης χωρικά ταχύτητας, οι οποίες προκύπτουν (σε `format long`) ίσες με:

$$V_{average}^{anal.} = 0.203718327157626 \quad (3.16a)$$

$$V_{average}^{num.} = 0.203687772753946 \quad (3.16b)$$

οι οποίες συμφωνούν (σε μορφή εκτύπωσης `format default`) έως και το τέταρτο δεκαδικό ψηφίο, δεδομένης της περιορισμένης ακρίβειας που προσφέρει η επιλογή αριθμού κόμβων στην τιμή $N = 101$ (100 διαστήματα διαμέρισης).

3. Σε αυτό το εδάφιο ελέγχεται η μεταβολή της αριθμητικής λύσης του υπό μελέτη προβλήματος υπό την μεταβολή του αριθμού των κόμβων του υπολογιστικού πλέγματος. Προς αυτό τον σκοπό, επιλέγονται οι εξής τιμές για τον αριθμό κόμβων:

$$N \in \{10, 20, 50, 100, 200, 500, 1000\} \quad (3.17)$$

και για καθεμία από αυτές, εκτελείται ο κώδικας που αναπτύχθηκε στα προηγούμενα εδάφια. Η διαδικασία αυτή πραγματοποιείται με το αρχείο τύπου m-script που παρατίθεται παρακάτω, η επεξήγηση του οποίου ακολουθεί την γενική λογική των προηγούμενων εδαφίων, και για αυτό παραλείπεται.

MATLAB SCRIPT M-FILE (GridModificationsEx3_1_Script.m):

```
clear
clc

% --- CFD Exercise 3.1 ---
% --- Part 3 ---
% Solution script (using ThomasAlgorithmTriInput.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of number of nodes N ---
N_values = [10, 20, 50, 100, 200, 500, 1000];
% --- Definition of analytical solution function handle ---
u_analytical = @(r) 2*(1-(r).^2);
% --- Main loop over N_values ---
for k = 1:length(N_values)
    N = N_values(k);
    % --- Definition of computational grid ---
    r = linspace(0,1,N);
    dr = 1/(N-1);
    % --- Definition of matrix A=diag(a,b,c) and d ---
    % Initializations:
    a = zeros(N,1) ;
    b = -2*ones(N,1); b(1) = 1; b(N) = 1;
    c = zeros(N,1) ; c(1) = -1; c(N) = 0;
    d = zeros(N,1) ;
    % Definitions:
    for i = 2:N-1
        a(i) = 1 - dr/(2*r(i));
        c(i) = 1 + dr/(2*r(i));
        d(i) = -8*(dr)^2;
    end
    %
    % -----
    % --- SOLUTION OF B.V.P. ---
    % -----
    % --- Apply Thomas algorithm and get solution ---
    u = ThomasAlgorithmTriInput(a,b,c,d);
    %
    % --- Definition of analytical solution at partition nodes ---
```

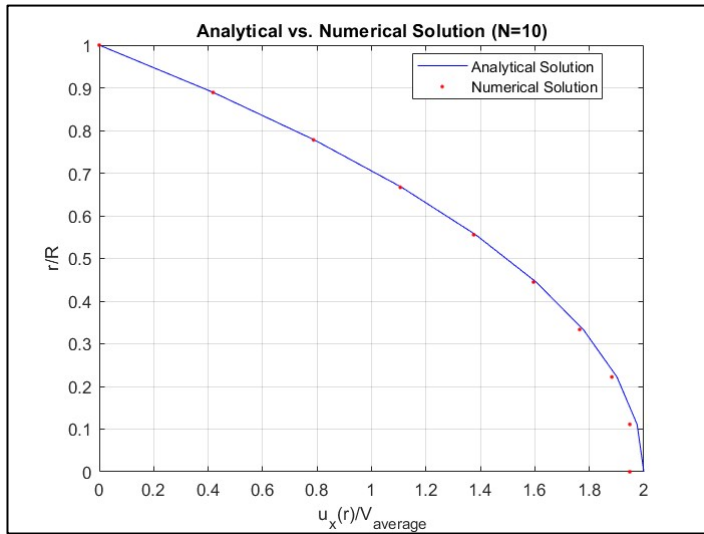
```

u_anal = u_analytical(r);
%
% --- Definition of absolute errors ---
absErrors = abs(u(:)-u_anal(:));
% --- Definition of table for data output ---
i = (1:N)';
T = table(i, r(:), u(:), u_anal(:), absErrors);
T = renamevars(T, "Var2", "r");
T = renamevars(T, "Var3", "u_numerical");
T = renamevars(T, "Var4", "u_analytical");
T = renamevars(T, "absErrors", "abs error");
sprintf('N=%d ', N)
disp(T(1:10,:));
disp(' .....');
disp(T(end-9:end,:));
%
% --- Plot numerical and analytical solution ---
u_figure = figure();
plot(u_anal, r, 'b-', 'LineWidth', 0.5); hold on;
plot(u, r, 'r.', 'LineWidth', 0.5);
grid on;
ylabel('r/R');
xlabel('u_{x}(r)/V_{average}');
title(['Analytical vs. Numerical Solution (N=', num2str(N), ')']);
legend('Analytical Solution', 'Numerical Solution', 'Location',
'Best');
%
% --- Plots absolute error for each grid node ---
err_figure = figure();
plot(absErrors, r, 'r.', 'LineWidth', 0.5);
grid on;
ylabel('r/R');
xlabel('Absolute Errors');
title(['Absolute Error for each node (N=', num2str(N), ')']);
legend('Absolute Error', 'Location', 'Best');
%
% -----
end
%
% End of script.

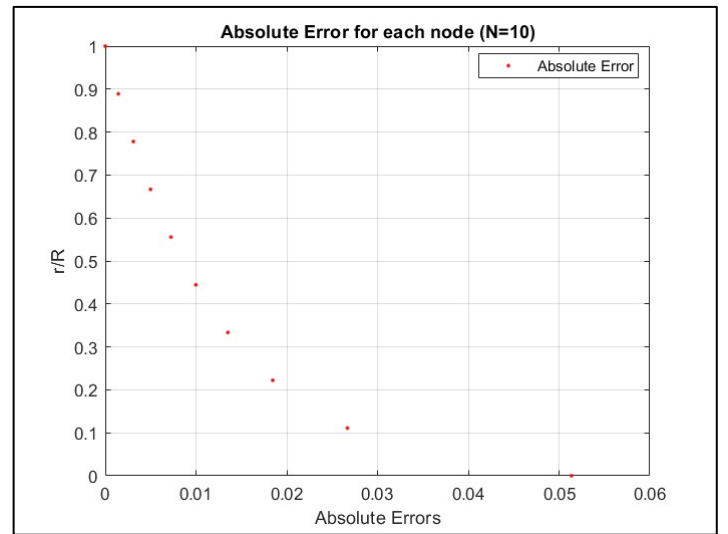
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του υπό μελέτη προβλήματος (στην αδιάστατη μορφή της), για κάθε τιμή του αριθμού, N , των κόμβων του υπολογιστικού πλέγματος, με μορφή εκτύπωσης που καθορίζεται από την συνάρτηση `table`. Μέσω του κώδικα του παραπάνω αρχείου, πραγματοποιείται, για κάθε τιμή του N , σχεδιασμός της γραφικής παράστασης της αριθμητικής και της αναλυτικής λύσης, καθώς και της γραφικής παράστασης των απολύτων σφαλμάτων μεταξύ αριθμητικής και αναλυτικής λύσης, σε κάθε κόμβο του υπολογιστικού πλέγματος. Τα προκύπτοντα διαγράμματα παρατίθενται παρακάτω, στα αντίστοιχα Διαγράμματα II έως VIII.

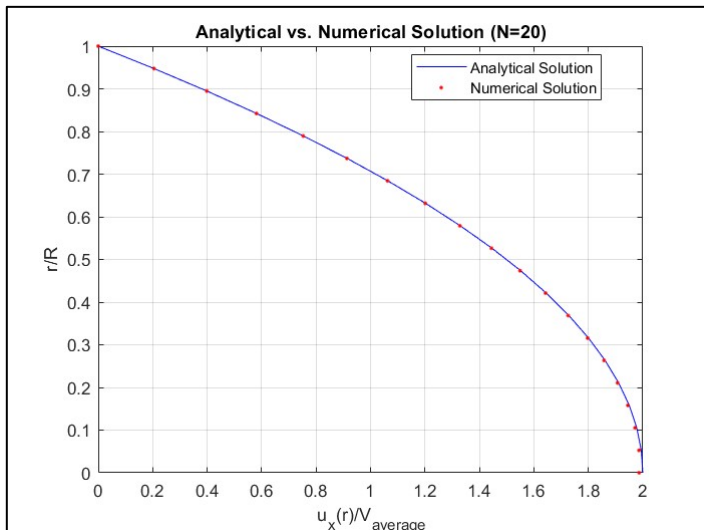
Είναι φανερό ότι καθώς το πλήθος, N , των κόμβων του υπολογιστικού πλέγματος αυξάνεται, η αριθμητική λύση αποτελεί όλο και καλύτερη προσέγγιση της αντίστοιχης αναλυτικής λύσης, καθώς το αντίστοιχο απόλυτο σφάλμα σε κάθε κόμβο μειώνεται.



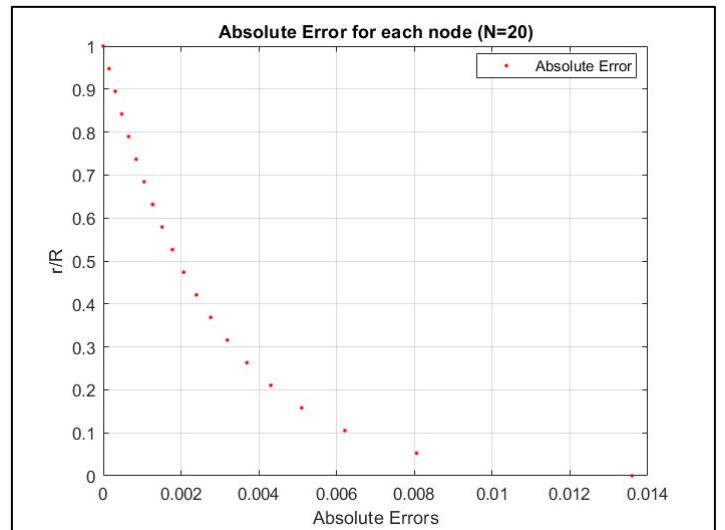
Διάγραμμα II(A).



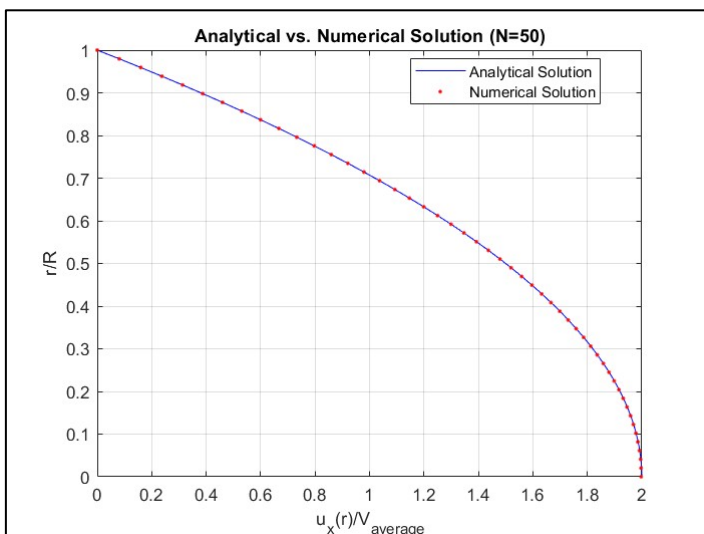
Διάγραμμα II(B).



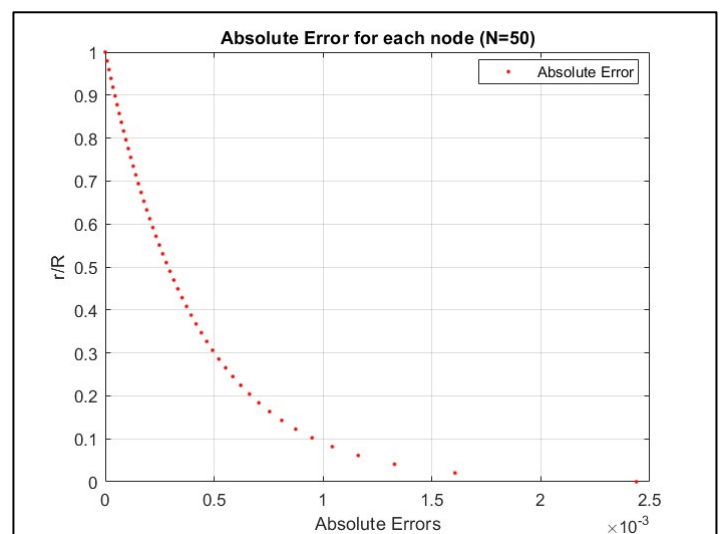
Διάγραμμα III(A).



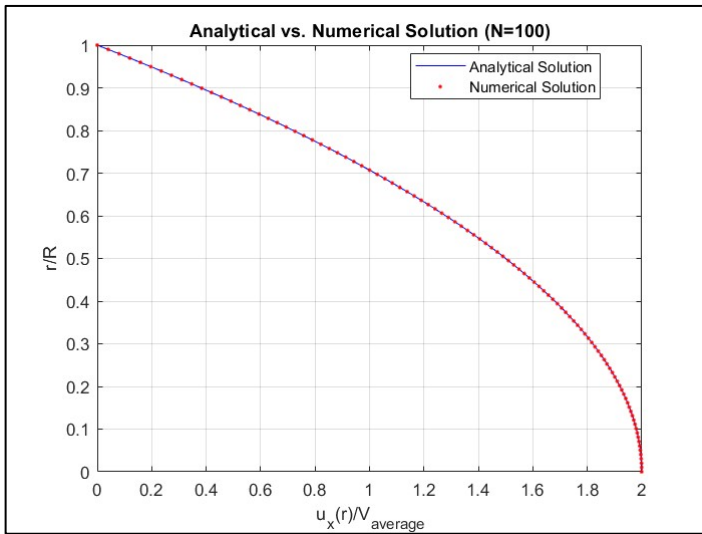
Διάγραμμα III(B).



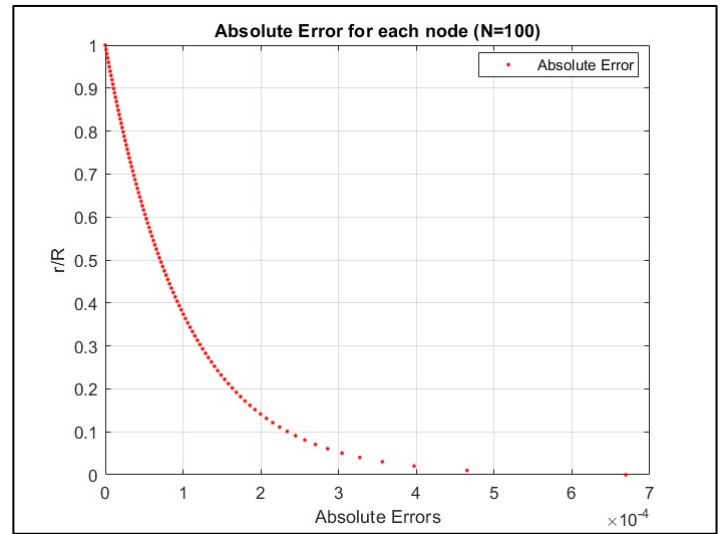
Διάγραμμα IV(A).



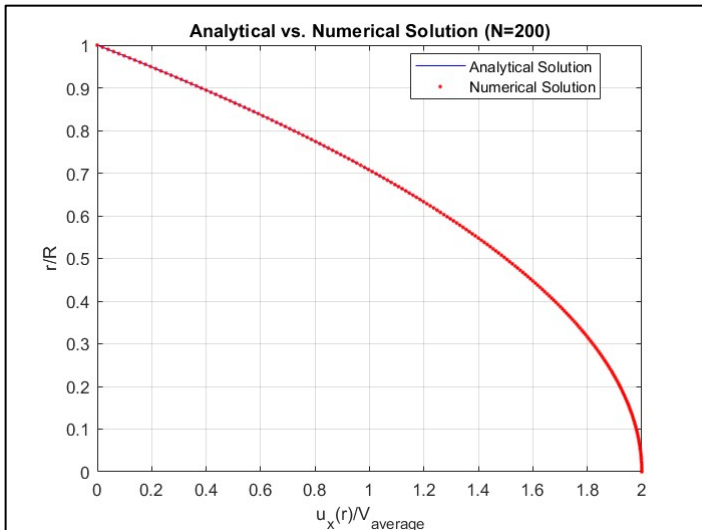
Διάγραμμα IV(B).



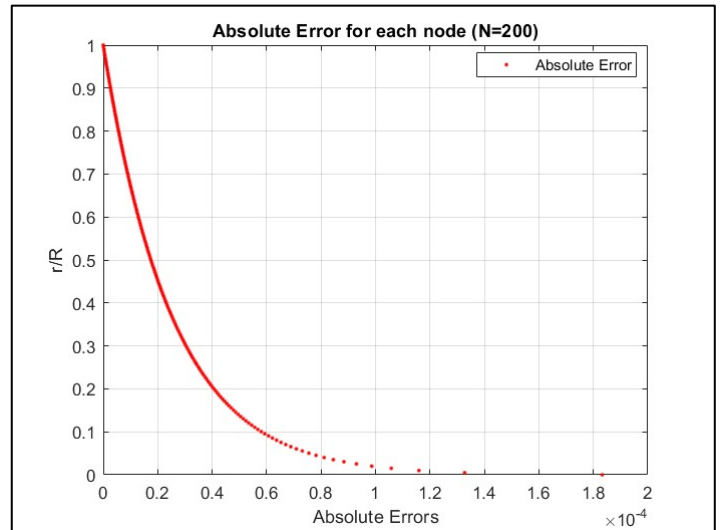
Διάγραμμα V(A).



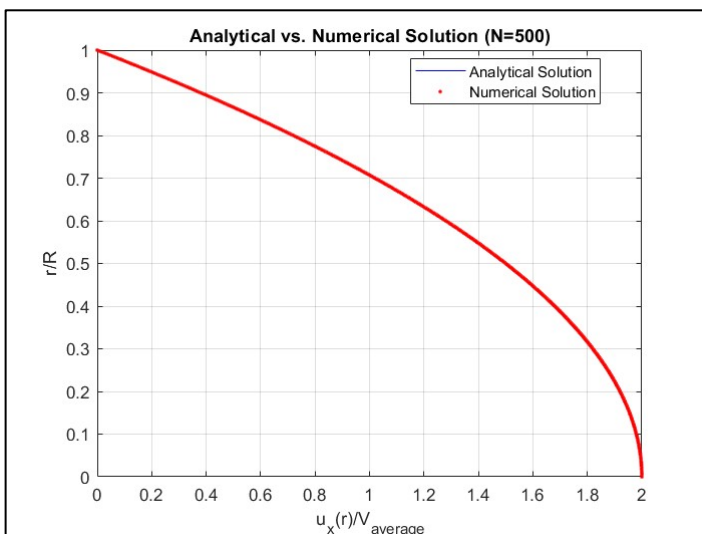
Διάγραμμα V(B).



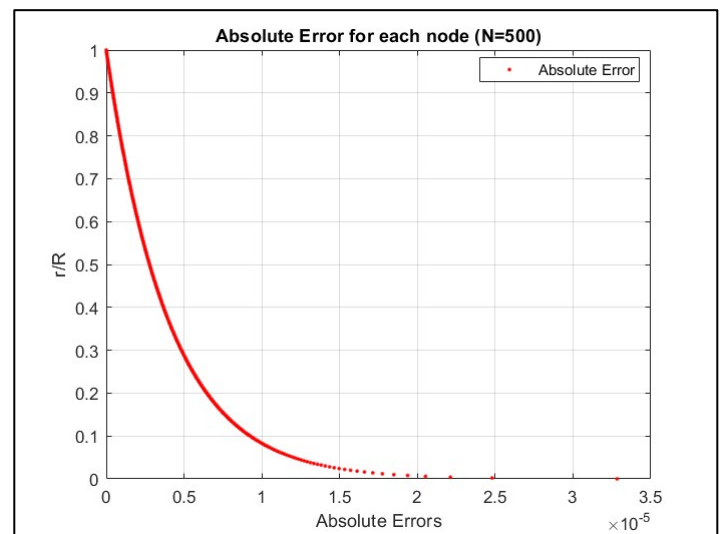
Διάγραμμα VI(A).



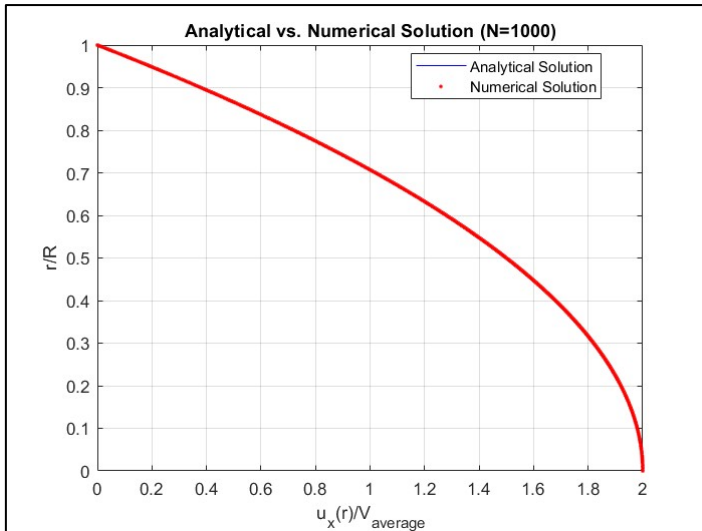
Διάγραμμα VI(B).



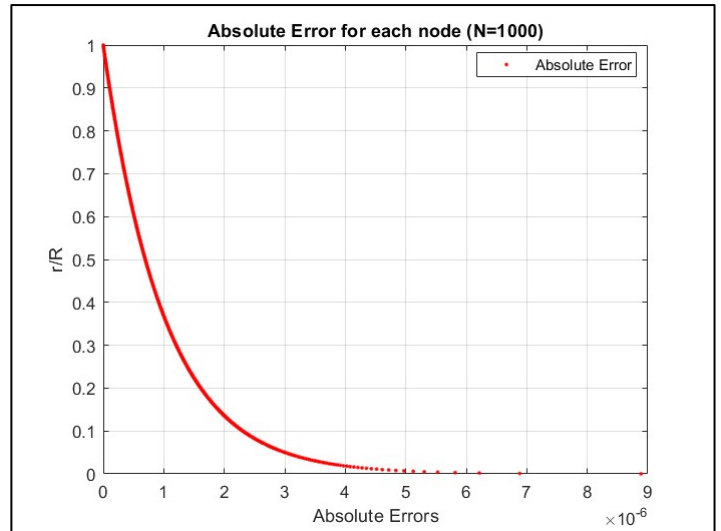
Διάγραμμα VII(A).



Διάγραμμα VII(B).



Διάγραμμα VIII(A).



Διάγραμμα VIII(B).

Προς ανάδειξη της τάξεως της υπό χρήση μεθόδου πεπερασμένων διαφορών, θα μπορούσε να ακολουθηθεί μία παρόμοια μέθοδος με αυτή του εδαφίου 6 της Άσκησης 2.1 της 2^{ης} Σειράς Ασκήσεων. Η μέθοδος πεπερασμένων διαφορών που υλοποιείται στα παραπάνω είναι ακρίβειας 1^{ης} τάξης, $O(\Delta r)$, λόγω του γεγονότος ότι η συνοριακή συνθήκη τύπου Newmann αντικαθίσταται από ακραία διαφορά πρώτης τάξεως, η ακρίβεια της οποίας μεταδίδεται και στους υπόλοιπους κόμβους. ■